

File Handling

Persistent Storage in C++



Mastering Data Persistence & Stream Classes

IFSTREAM • OFSTREAM • FSTREAM

1. Beyond the RAM

Normally, data in a C++ program is stored in RAM and lost when the program closes. **File Handling** allows us to store data on the hard drive permanently.

The Library: `#include <fstream>`

This library provides three main classes for handling files, much like the `iostream` library provides classes for console I/O.

Class	Description
ofstream	Output File Stream: Used to create and write files.
ifstream	Input File Stream: Used to read files.
fstream	File Stream: General-purpose class for both read and write.

2. Creating & Writing

Think of `ofstream` as a replacement for `cout`. Instead of sending text to the screen, you send it to a file on your disk.

```
#include <fstream>

int main() {
    // 1. Open the file
    std::ofstream myFile("notes.txt");

    // 2. Write to it (Just like cout)
    myFile << "Persistent Data Practice
";
    myFile << "End of line.";

    // 3. Close it (Very Important!)
    myFile.close();
}
```

3. Retrieving Data

The `ifstream` class acts like `cin`. We use a loop to pull data out of the file until we reach the end.

```
#include <string>
#include <fstream>

int main() {
    std::ifstream readingFile("notes.txt");
    std::string textLine;

    // Use getline to read whole lines
    while (getline(readingFile, textLine)) {
        std::cout << textLine << std::endl;
    }

    readingFile.close();
}
```

4. Advanced Open Modes

When opening a file, you can specify **how** you want to open it using flags from the `ios` namespace.

```
fstream file("data.bin", ios::out | ios::binary);
```

Flag	Action
<code>ios::in</code>	Open for reading.
<code>ios::out</code>	Open for writing (overwrites existing).
<code>ios::app</code>	Append: Add new data to the end.
<code>ios::ate</code>	At the end: Open and move pointer to end.
<code>ios::binary</code>	Open in binary mode.

5. Don't Overwrite: Append

By default, `ofstream` wipes the file clean every time you open it. To keep old data and add to it, use the `app` flag.

```
// This will NOT delete previous content
ofstream myLog("log.txt", ios::app);

myLog << "New Log Entry: " << __TIME__ << "
";
myLog.close();
```

Analogy: Overwriting is like getting a new notebook every day. Appending is like continuing on the next blank page of your current notebook.

6. Defensive Programming

Files might be missing, locked by other programs, or corrupted. Always check if the file opened successfully before attempting to read/write.

```
ifstream checkFile("config.txt");

if (!checkFile) {
    cerr << "Fatal Error: File could not be found!" << endl;
    return 1; // Exit program
}

if (checkFile.is_open()) {
    // Proceed with logic
}
```

7. Combined Streams (fstream)

The `fstream` class is the Swiss Army Knife of file handling. It can switch between reading and writing without needing separate objects.

```
fstream masterFile;  
masterFile.open("database.txt", ios::in | ios::out);  
  
// Use logic to read some parts  
// and write others ...
```

Note: When using `fstream` for both, you often need to manage the "File Pointer" (where the cursor is currently resting) using `seekg()` and `seekp()`.

8. File Handling Etiquette

1. Clear Your Buffers

Data isn't always written immediately to the disk; it stays in a "buffer." Closing the file or using `flush()` ensures the data is physically saved.

2. Text vs Binary

Text files are human-readable. Binary files are faster and smaller but look like gibberish to humans. Use Binary for images, saves, and complex data structures.

3. Path Handling

Remember that `"data.txt"` looks in the same folder as your EXE. For absolute paths, use double backslashes: `"C:\\Users\\Data\\file.txt"`.

9. Summary Checklist

- **Include:** `<fstream>`.
- **Write:** Use `ofstream`.
- **Read:** Use `ifstream`.
- **Validate:** Check `if(!file)`.
- **Finalize:** Always call `.close()`.

Skill Lab: Student Registry

The Task: Create a program that asks for a student's name and marks. Save this data to a file called `students.txt` using **append mode**. Then, write a second function that reads the file and calculates the average score of everyone in the list.

STORAGE VERIFIED: FILE HANDLING MASTERED